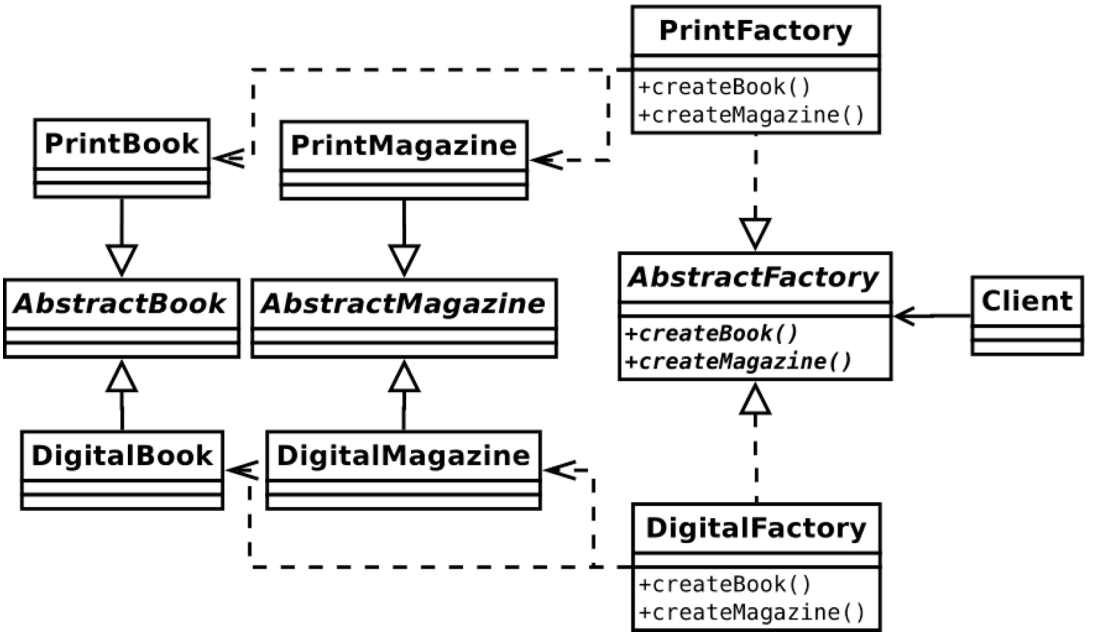


Figure 10-19. *Abstract Factory pattern*

Things get even more complex when dealing with objects that are composed of other objects (e.g., the structural pattern known as *Composite*, which is discussed below) or needing to control the number of instances of a class. Naturally there are creational patterns to suit those cases: the other main types are

- Builder (creates composite objects)
- Singleton (ensures only a single instance of an object exists globally)
- Prototype (creates copies of objects without introducing additional dependencies)

We won't look at these in detail here; detailed information is readily available online or in the aforementioned books. Also worth mentioning at this point are some patterns that are adjacent to object creation. By now you should be comfortable with the concept of dependency: objects and functions often rely on other objects/functions/data for their correct operation. There are patterns for managing that too, and they are collectively known as *Dependency Injection*. Two common patterns are: